C Language:

History of C

- Evolved by Dennis Ritchie from two previous programming languages, BCPL and B.
- Used to develop UNIX .
- Used to write modern operating systems.
- Hardware independent (portable).

Introduction to C:

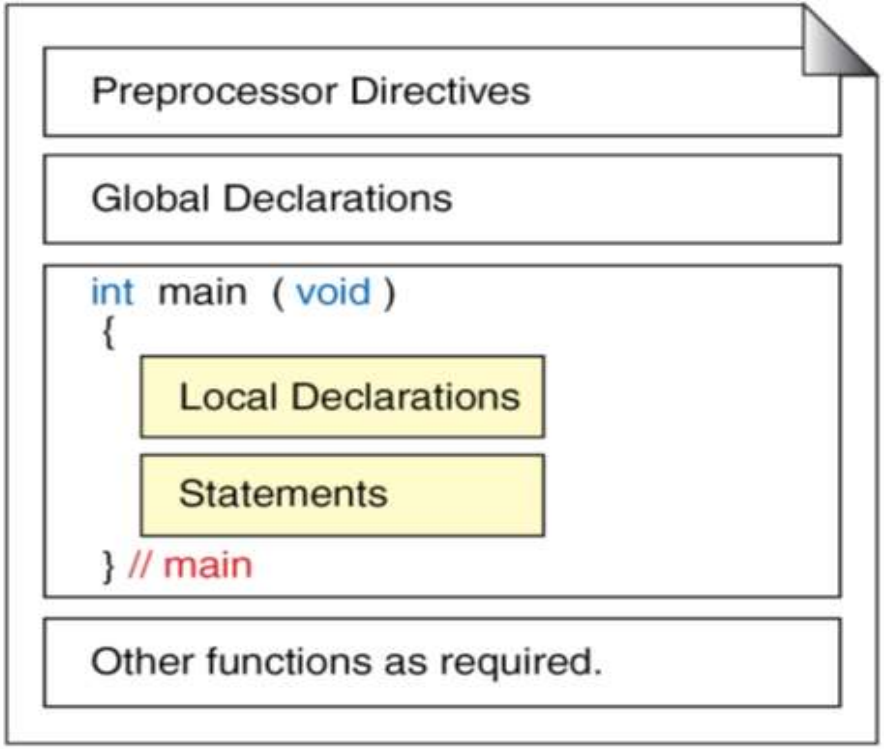
C is a Middle-level language: suitable language for systems programming

- It is a procedural language
- Very close to the hardware resources of the computer
- C is easy to learn or understand
- It is a case sensitive language
- C is a small language: relies on a “library” of standard functions
- C is highly portable: means that c programs written for one computer can be run on other computer with no modification
- It can handle low-level activities.
- It can be compiled on a variety of computer platforms.

Features of C Language:

- Portability:
- Robustness:
- Easy to understand:
- Efficiency:
- Power:
- Flexibility:
- Programs written in C is fast and efficient .
- Standard library.

Structure of C:



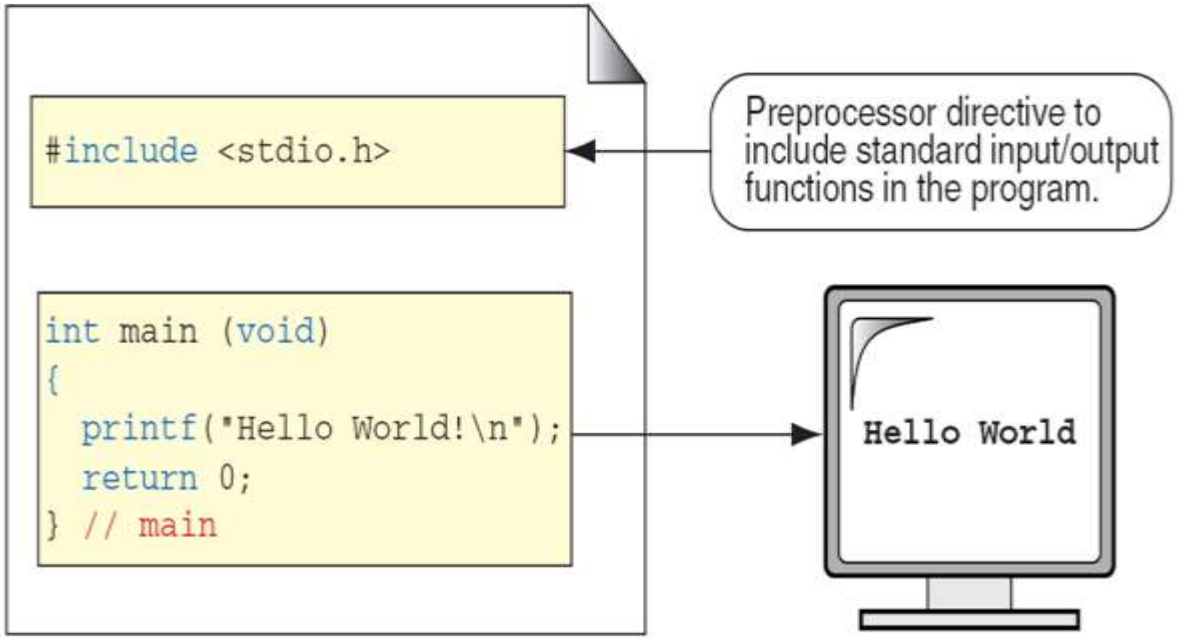
STEPS TO WRITE C PROGRAMS AND GET THE OUTPUT:

Below are the steps to be followed for any C program to create and get the output. This is common to all C program and there is no exception whether its a very small C program or very large C program.

- Create
- Compile
- Execute or Run
- Get the Output

Simple C Program:

```
/* This is the sample program to print a message hello world.
This is done by course teacher */
 #include
#include
main ( )
 {
clrscr();
printf(“Hello World\n”);
getch();
}
```



Header Files Inclusion:

- The first foremost component is the Header files in a C program.
- A header file is a file with extension .h which contains C function declarations and macro definitions to be shared between several source files.

- Some of C Header files:
  - stdint.h – Defines exact width integer types.
  - stdio.h – Defines core input and output functions
  - string.h – Defines string handling functions
  - math.h – Defines common mathematical functions

Syntax to include a header file in C:

#include

All lines that start with # are processed by pre-processor which is a program invoked by the compiler. Pre-processor copies the pre-processed code of stdio.h to our file. The .h files are called header files in C. These header files generally contain declaration of functions. We need stdio.h for the function printf() used in the program.

Main Method Declaration:

The next part of a C program is to declare the main () function. The syntax to declare the main function is:

Syntax to Declare main method:

```
int main()
{
}
```

C program is executed in three phases:

- Creating Source Code file:

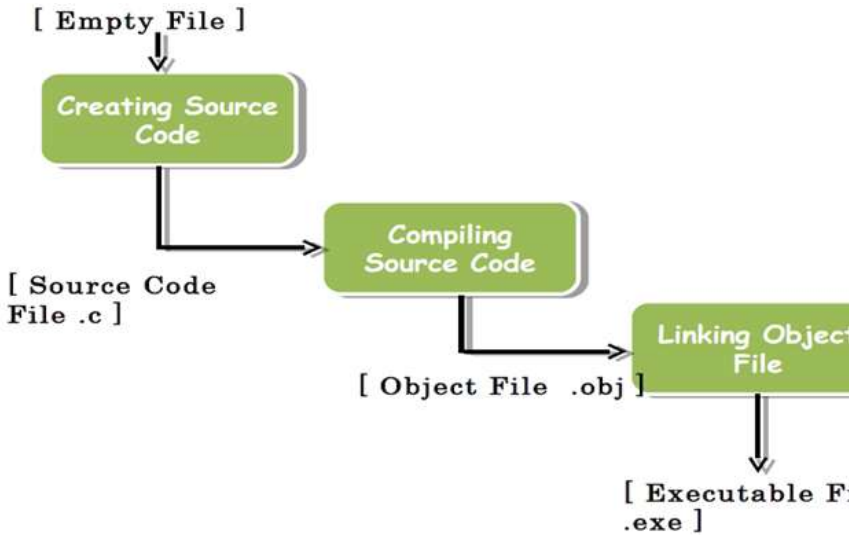
  [Empty file(input) ➔ Source Code File(output)]

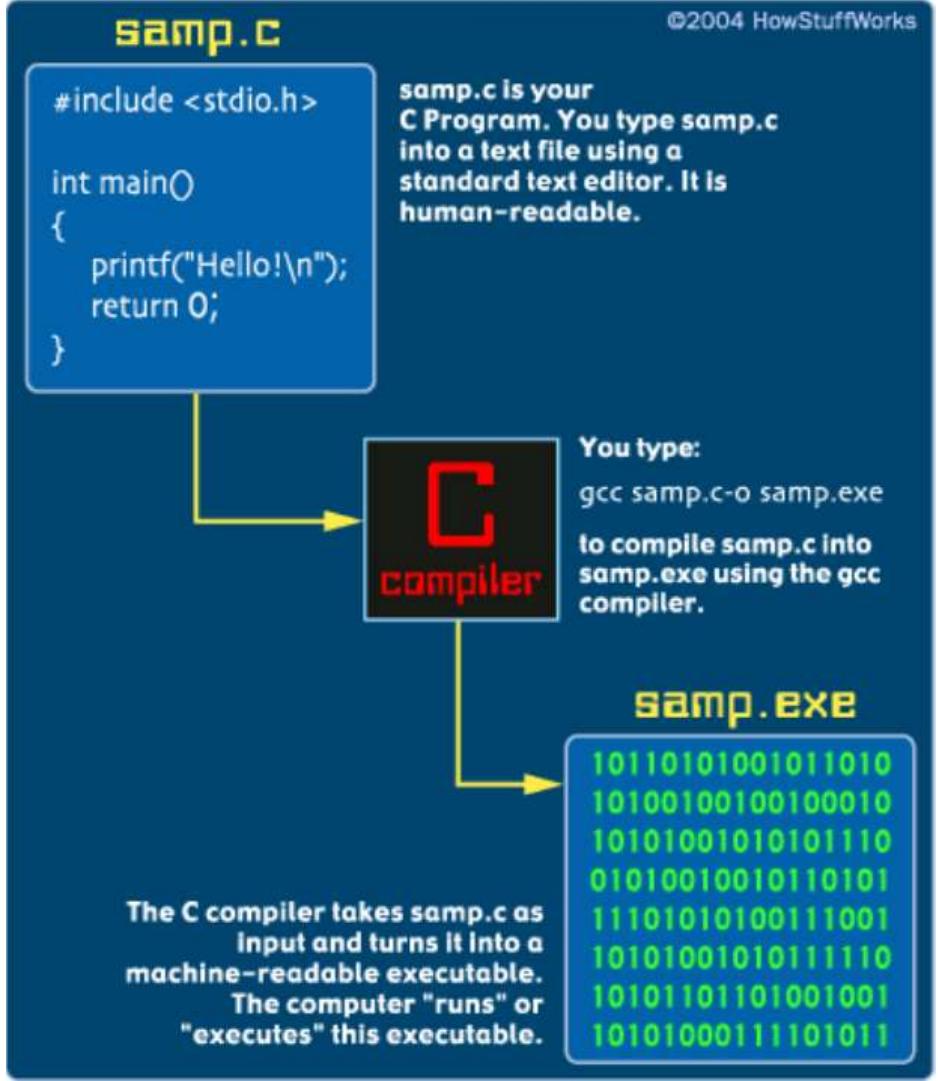
- Compiling:

  [Source Code File(input) ➔ Object File(output)]

- Linking:

  [Object File(input) ➔ Executable File(output)]




samp.c
©2004 HowStuffWorks
#include <stdio.h>
int main()
{
printf("Hello!\n");
return 0;
}
samp.c is your C Program. You type samp.c into a text file using a standard text editor. It is human-readable.
C
compiler
You type:
gcc samp.c-o samp.exe
to compile samp.c into samp.exe using the gcc compiler.
samp.exe
10110101001011010
10100100100100010
10101001010101110
01010010010110101
11101010100111001
10101001010111110
10101101101001001
10101000111101011
The C compiler takes samp.c as input and turns it into a machine-readable executable. The computer "runs" or "executes" this executable.

Variable Declaration:

- Variables that are to be used in the function. In C program, no variable can be used without being declared. Variables are to be declared before any operation in the function.

Example:

```
int main()
{
    int a;
}
```

Body:

Body of a function in C program, refers to the operations that are performed in the functions. It can be anything like manipulations, searching, sorting, printing, etc.

Example:

```
int main()
{
    int a;
    printf("%d", a);
}
```

PRINTF() FUNCTION IN C LANGUAGE:

- In C programming language, printf() function is used to print the (“character, string, float, integer, octal and hexadecimal values”) onto the output screen.
- We use printf() function with %d format specifier to display the value of an integer variable.
- Similarly %c is used to display character, %f for float variable, %s for string variable, %lf for double and %x for hexadecimal variable.
- To generate a newline,we use “\n” in C printf() statement.

```c
#include <stdio.h>
int main()
{
    char ch = 'A';
    char str[20] = "fresh2refresh.com";
    float flt = 10.234;
    int no = 150;
    double dbl = 20.123456;
    printf("Character is %c \n", ch);
    printf("String is %s \n" , str);
    printf("Float value is %f \n", flt);
    printf("Integer value is %d\n" , no);
    printf("Double value is %lf \n", dbl);
    printf("Octal value is %o \n", no);
    printf("Hexadecimal value is %x \n", no);
    return 0;
}
```

OUTPUT:

```
Character is A
String is fresh2refresh.com
Float value is 10.234000
Integer value is 150
Double value is 20.123456
Octal value is 226
Hexadecimal value is 96
```

NOTE: C language is case sensitive. For example, printf() and scanf() are different from Printf() and Scanf(). All characters in printf() and scanf() functions must be in lower case.

SCANF FUNCTION IN C LANGUAGE:

- In C programming language, scanf() function is used to read character, string, numeric data from keyboard
- Consider below example program where user enters a character. This value is assigned to the variable “ch” and then displayed.

EXAMPLE PROGRAM FOR C PRINTF AND SCANF FUNCTIONS IN C PROGRAMMING LANGUAGE:

```c
#include <stdio.h>
int main()
{
    char ch;
    char str[100];
    printf("Enter any character \n");
    scanf("%c", &ch);
    printf("Entered character is %c \n", ch);
    printf("Enter any string ( upto 100 character ) \n");
    scanf("%s", &str);
    printf("Entered string is %s \n", str);
}
```

OUTPUT :

Enter any character

a

Entered character is a

Enter any string (upto 100 character)

hai

Entered string is hai

- The format specifier %d is used in scanf() statement. So that, the value entered is received as an integer and %s for string.
- Ampersand is used before variable name “ch” in scanf() statement as &ch.
- It is just like in a pointer which is used to point to the variable.

Return Statement:

Return statement refers to the returning of the values from a function. This return statement and return value depend upon the return type of the function. If the return type is void, then there will be no return statement.

```
int main()
{
        int a;

    printf("%d", a);

        return 0;
}
```

The return statement returns the value from main(). The returned value may be used by operating system to know termination status of your program. The value 0 typically means successful termination.

Tokens in C:

A C program consists of various tokens and a token is either a keyword, an identifier, a constant, a string literal, or a symbol. For example, the following C statement consists of five tokens −

The ‘C’ tokens are classified as:

- Character set
- Keywords.
- Identifiers.
- Constants.
- Operators.

Character set:

Character set consists of

- Alphabet from A-Z or a-z.
- Digits from 0-9.
- Special Characters like (,), {,}, [,], <, >, &, $, #, %, ^, !, ?, :, ;, ",',
- White space character: blank space Escape sequences:

\b	backspace
\a	audible bell
\v	vertical tab
\t	horizontal tab
\f	form feed
\r	carriage return
\”	double quotes
\’	single quotes
\\	back slash

Keywords:

Keywords are reserved words. All keywords have fixed meanings. Keywords serve as basic building blocks for program statements. In ‘C’, there are 32 keywords. All keywords must be written in lower case.

These 32 keywords are classified into 3 types namely

- Type related Keywords (16).
- Storage related Keywords (4).
- Control flow related Keywords (12)

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	Volatile
const	float	short	Unsigned

Identifiers:

Identifier refers to the names of the variables, functions and arrays.

Rules to name a particular identifier:

- It must start with either alphabet or underscore.
- Remaining letters may be alphabet, digit, underscore.
- Identifier would not allow any special symbol except underscore.
- An identifier can be of any length while most compilers of ‘C’ language recognize only the first 8 characters.
- Do not use keywords as an identifier.

Constants:

- Constants refer to fixed values that do not change during the execution of a program.

Constants are 2 types.

1.Numerical constants

2.Non-numerical constants

- Numerical Constants are 2 types.
  - Integer constants
  - Real constants

- Non- Numerical Constants are 2 types.
  - Character constants
  - String or multiple character constants

Integer constants:

An integer constant refers to a sequence of digits.

There are three types of integers, namely, decimal, octal and hexadecimal.

- Decimal integers consist of a set of digits, 0 through 9, preceded by an optional – or + sign.
- Valid examples of decimal integer constants are: 321, -789, 0, +77.
- An octal integer constant consists of any combination of digits from the set 0 through 7, with leading 0.
- Some examples of octal integers are 047, 0, 0543, 0655.
- Hexadecimal integer constant consists of a sequence of digits preceded by 0x or 0X.
- They may also include alphabets A through F or a through f. (0 – 9 and A – F or a – f).

Examples of hexadecimal integer constants are: 0X49, 0x8D

Real constants:

- Some quantities vary continuously, such as distances, heights, temperatures, prices and so on.
- These quantities are represented by numbers containing fractional parts. Such numbers are called real or floating point constants.
- A real constant is a sequence of digits with a decimal point.
- Examples of real constants are: 0.00876, -0.456, 543.78, +247.0.
- A real number may also be expressed in exponential or scientific notation.

Example is: The value 289.45 may be written as 2.8945e2 in exponential notation.

Single Character constants (Character constant):

A single character constant contains a single character (alphabet, digit, special symbol, white space) enclosed within a pair of single quote marks.

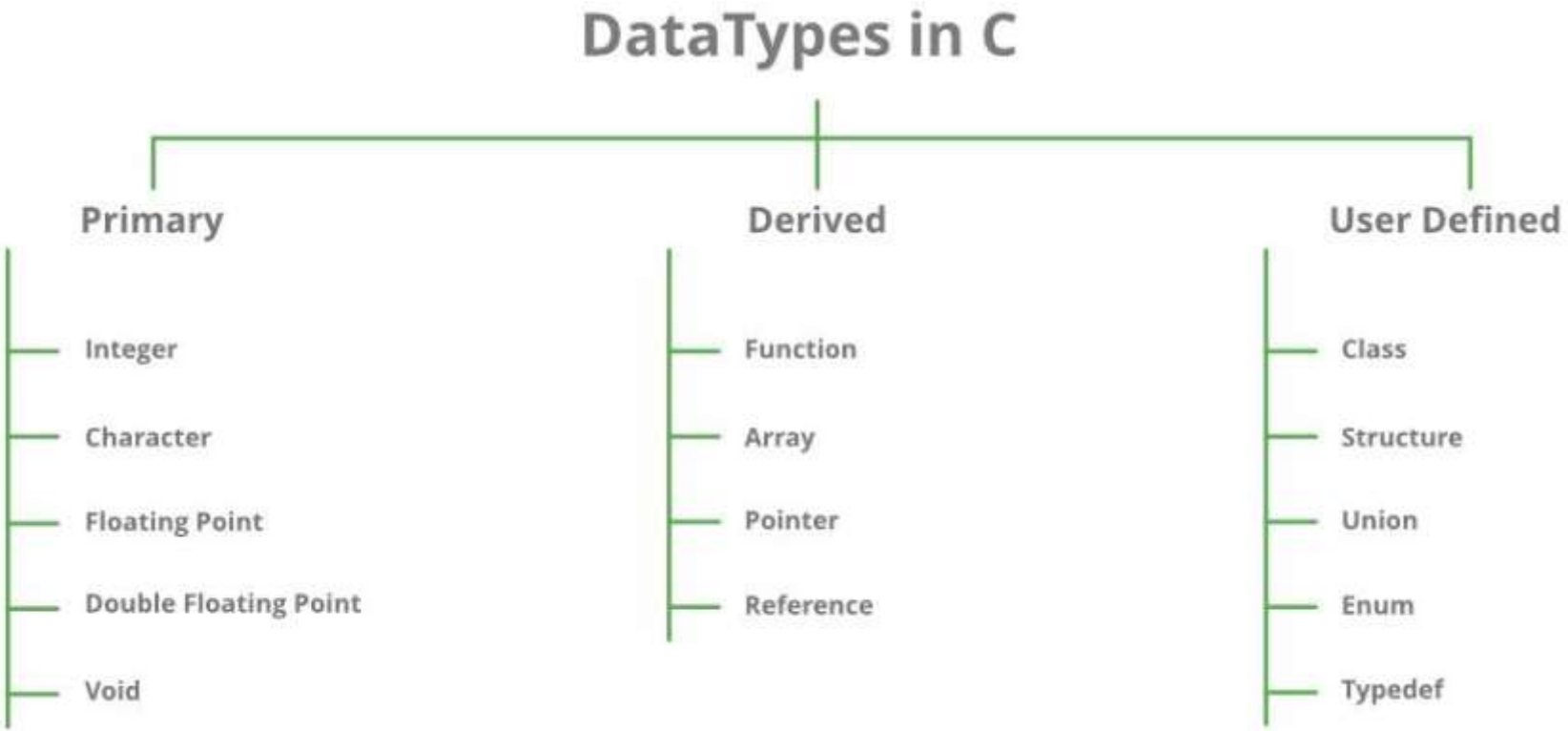
Examples are: '1', 'a', ' ','@'

Example Program:

```
#include<stdio.h>
main()
{
char ch='a';
printf("ch=%c\n",ch);
printf("ASCII value of \'%c\'=%d\n",ch,ch);
}
Output:
ch=a
ASCII value of 'a'=97
```

String constants (Multiple characters):

- String is a sequence of characters enclosed in double quotes.
- The characters may be an alphabet, digit, special symbol and white space.
- Each and every string ends with null character '\0'.
- Examples are: "ab", "ab5", "@an", "6*7"
- Variable is a data name which can be used to store a data and a variable may take different values at different times during execution.


DataTypes in C
Primary
Integer
Character
Floating Point
Double Floating Point
Void
Derived
Function
Array
Pointer
Reference
User Defined
Class
Structure
Union
Enum
Typedef

Data types:

Data type specifies which type of data that we are storing in a variable.

There are 3 types of data types.

- Primary or primitive or fundamental data types
- User defined data types.
- Derived data types
- Empty data set

Primary data types: There four fundamental data types.

- Integer.
- Floating point
- Character
- Double precision floating point.

Integer:

'C' provides three different classes of integers.

- Int
- short int
- long int

Th ies.

Type	Bytes	Bits	Range
signed short int or short int	2	16	-32768 to 32767
signed int or int	2/4	16/32	-32768 to 32767 or -2147483648 to 2147483647
signed long int or long int	4	32	-2147483648 to 2147483647
unsigned short int	2	16	0 to 65535
unsigned int	2/4	16/32	0 to 65535 or 0 to 4294967295
unsigned long int	4	32	0 to 4294967295

Classes of floating point types:

'C' provides three different classes of float.

- Float
- Double
- long double

Type	Bytes	Bits	Range	Precision
float	4	32	3.4E-38 to 3.4E+38	6 bits
double	8	64	1.7E-308 to 1.7E+308	14 bits
long double	12	96	3.4E-4932 to 1.1E+4932	19 bits

Character types:

A single character can be defined as a character type data.

Type	Bytes	Bits	Range
signed char	1	8	-128 to 127
unsigned char	1	8	0 to 255

Operators:

- This is a symbol use to perform some operation on variables, operands or with the constant. Some operator required 2 operand to perform operation or Some required single operation.
- Several operators are there those are:

1) Arithmetic Operator.
2) Assignment Operator.
3) Increment Operator.
4) Decrement Operator.
5) Logical Operator.
6) Conditional Operator.
7) Comma.
8) size of Operator.
9) Bitwise Operator, and others.

Arithmetic Operator:

- This operator used for numeric calculation.
- These are of either Unary arithmetic operator, Binary arithmetic operator.

Binary arithmetic operator on other hand required two operand and its operators are +(addition), -(subtraction),*(multiplication), /(division), %(modulus).

Modulus cannot applied with floating point operand as well as there are no exponent operator in c.

Unary (+) and Unary (-) is different from addition and subtraction. When both the operand are integer then it is called integer arithmetic and the result is always integer. When both the operand are floating point then it is called floating arithmetic and when operand is of integer and floating point then it is called mix type or mixed mode arithmetic . And the result is in float type.

Assignment Operator:

- A value can be stored in a variable with the use of assignment operator. The assignment operator(=) is used in assignment statement and assignment expression.
- Operand on the left-hand side should be variable and the operand on the right hand side should be variable or constant or any expression. When variable on the left hand side is occur on the right hand side then we can avoid by writing the compound statement.

For example,

```
    int x= y;
int Sum=x+y+z;
```

Increment Operator in C:

Unary arithmetic operator required only one operand such as +,-, ++, --,!.
The **increment operator (++)** is used to increment the value of a variable in an expression by 1. It can be used on variables of the numeric type such as integer, float, character, pointers, etc.

Syntax of Increment Operator

Increment Operator can be used in two ways which are as follows:

```
// AS PREFIX
++i

// AS POSTFIX
i++
```

Decrement Operator in C:

The decrement operator is used to decrement the value of a variable in an expression. In the Pre-Decrement, the value is first decremented and then used inside the expression. Whereas in the Post-Decrement, the value is first used inside the expression and then decremented.

Syntax

Just like the increment operator, the decrement operator can also be used in two ways:

```
// AS PREFIX

--i

// AS POSTFIX

  i--
```

Relational Operator:

It is use to compared value of two expressions depending on their relation. Expression that contain relational operator is called relational expression.

Here the value is assigned according to true or false value.

- >
- >=
- <
- <=
- ==
- !=

Conditional Operator:

- It sometimes called as ternary operator. It requires three expressions as operand and it is represented as (?:).

Syntax:

cond? exp1: exp2;

Here cond is first evaluated. It is true then value return will be exp1 . If false then exp2.

Example:

```
void main ()
{
int a=10, b=2int
s= (a>b) ? a:b;
printf("Max:%d",s);
}
```

Output:

Max:106.

Comma Operator:

- Comma operator is use to permit different expression to be appear in a situation where only one expression would be used. All the expression are separated by comma and are evaluated from left to right.

Example :

```
int i, j, k, l;

for(i=1,j=2;i<=5;j<=10;i++;j++)
```

Logical or Boolean Operator:

- Operator used with one or more operand and return either value zero (for false) or one (for true). The operand may be constant, variables or expressions. And the expression that combines two or more expressions is termed as logical expression.

C has three logical operators :

Operator	Meaning
&&	AND
||	OR
!	NOT

Bitwise Operator:

Bitwise operator permit programmer to access and manipulate of data at bit level.

Various bitwise operator enlisted are:

- one's complement ~
- bitwise AND &
- bitwise OR |
- bitwise XOR ^
- left shift <<
- right shift >>

These operator scan operate on integer and character value but not on float and double.

In one's complement all 0 changes to 1 and all 1 changes to 0. In the bitwise OR its value would obtaining by 0 to 2 bits.

As the bitwise OR operator is used to set on a particular bit in a number.

Sizeof Operator:

- Size of operator is a Unary operator, which gives size of operand in terms of byte that occupied in the memory. An operand may be variable, constant or data type qualifier.

Generally it is used make portable program(program that can be run on different machine) .

- It determines the length of entities, arrays and structures when their sizes are not known to the programmer. It is also use to allocate size of memory dynamically during execution of the program.

EXAMPLE:

```
main( )
    {
    int sum;
    float f;
    printf( "%d%d" ,sizeof(f), sizeof (sum) );
    }
```

What is Type casting in C?

In C, When you convert the data type of a variable to another data type then this technique is known as typecasting.
Let’s say that you want to store a value of int data type into a variable of float data type.

Then you can easily do this with the help of typecasting. It is one of the important concepts of the C programming language.

Types of Type casting in C:-

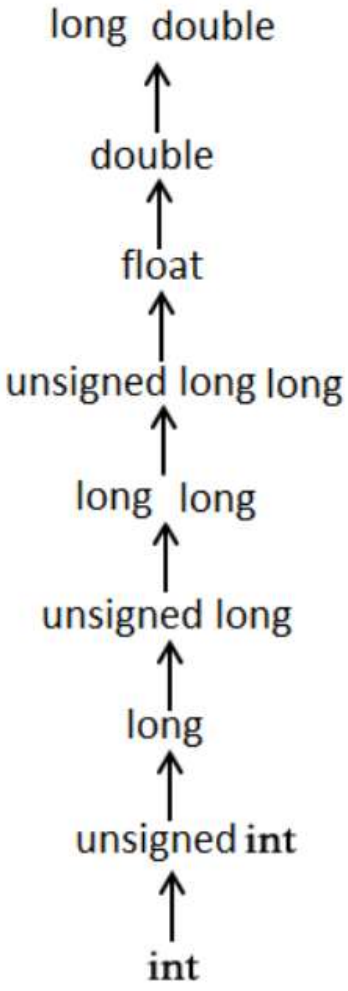
There are 2 types of type casting operations present in C such as:-

- Implicit Type Casting
- Explicit Type Casting

1. Implicit Type Casting in C

It is very easy to implement implicit type casting. With the help of implicit type casting, you can convert a data type of a variable into another data type without losing its actual meaning.

It will perform the conversion without changing the importance of the values stored inside the variable.


long double
double
float
unsigned long long
long long
unsigned long
long
unsigned int
int

2. Explicit Type casting in C

It is not like implicit type casting where the data type is converted automatically. In Explicit type casting, you have to force the conversion between data types.

Syntax:-

```
(data_type_name) expression
```

- data_type_name is the name of the data type to which you want to convert to.
- The expression can be a variable, an actual expression or a constant.

Control statements in C:

IF STATEMENT:

```
if (test expression)
{
statement/s to be executed if test
expression is true;
}
```

C IF...ELSE STATEMENT:

Syntax of if...else

```
if (test expression)
{
statements to be executed if test expression is true;
}
else
{
statements to be executed if test expression is false;
}
```

Nested elseif():

```
if (test expression1){
statement/s to be executed if test expression1 is true;
}
else if(test expression2) {
statement/s to be executed if test expression1 is false and 2 is true;
}
else if (test expression 3) {
statement/s to be executed if text expression1 and 2 are false and 3 is true;
}
.
```

C PROGRAMMING LOOPS :

A loop statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages.

C programming language provides the following types of loop to handle looping requirements.

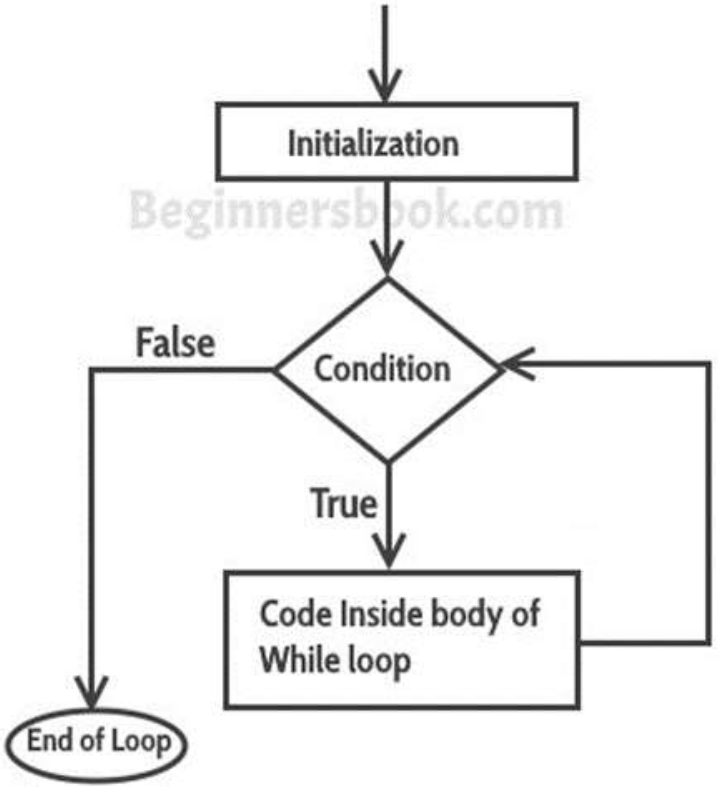
- while loop.
- for loop.
- do...while loop.

WHILE LOOP:

- The while loop in c is to be used in the scenario where the block of statements is executed in the while loop until the condition specified in the while loop is satisfied.
- The “while” statement is used to carry out looping operations, in which a group of statements is executed repeatedly, as long as some condition remains satisfied.

Syntax of while loop in c language is given below:

```
initialization;
while(condition)
{
block of statements to be executed ;
Increment / decrement;
}
```



FOR LOOP:

The **for loop in C language** is used to iterate the statements or a part of the program several times.

Syntax:

```
for(initialization; condition; Inc/Dec)
{
codes to be executed;
}
```

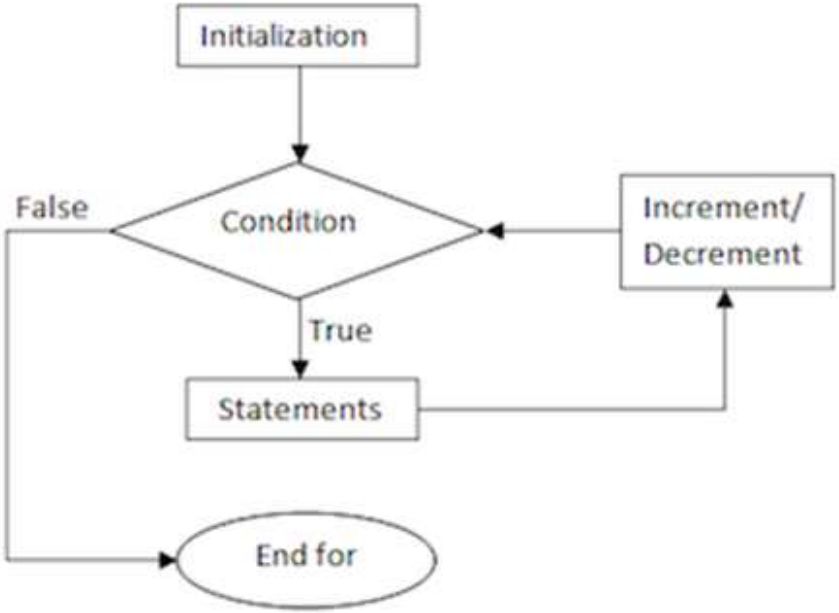
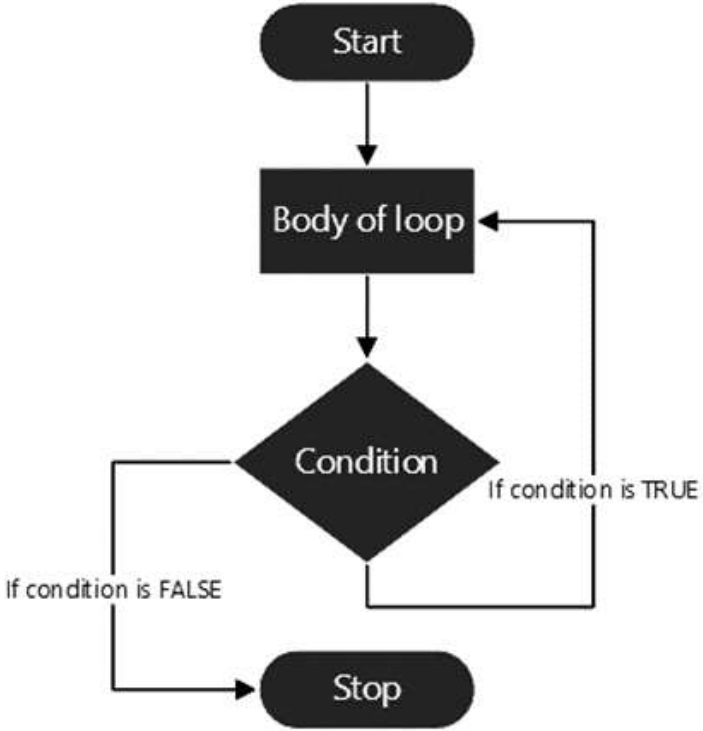


fig: Flowchart for for loop

DO WHILE LOOP:

- The ‘do...while’ loop, which executes a block of statements as long as a specified condition is true.
- The do...while loop tests the condition at the end of the loop rather than at the beginning, as is done by the ‘for’ loop and the while loop.
- Condition is any C expression, and statement is a single or compound C statement.



Syntax:

initialization;

do

{

code to be executed ;

Inc / dec;

}

while(condition);

ARRAY:

- An array in C Programming can be defined as number of memory locations, each of which can store the same data type and which can be references through the same variable name.
- It is a collective name given to a group of similar quantities.
- Array is a sequence of data item of homogeneous values (same type).
- These values could be all integers, floats or characters etc.

For example, if you want to store 100 integers, you can create an array for it.

Declaration:

dataType arrayName[arraySize];

int data[100];

We have two types of arrays:

- One-dimensional arrays.
- Multidimensional arrays.

One Dimensional Arrays:

Conceptually you can think of a one-dimensional array as a row, where elements are stored one after another.

• An array variable must be declared before being used in a program.

• The declaration must have a data type(int, float, char, double, etc.), variable name, and subscript.

• The subscript represents the size of the array. If the size is declared as 10, programmers can store 10 elements.

• An array index always starts from 0. For example, if an array variable is declared as s[10], then it ranges from 0 to 9.

• Each array element stored in a separate memory location.

Declaration syntax of one dimensional array:

data_type array_name[array_size];

• “data_type” is the type of the array we want to define,

• “array_name” is the name given to the array and

• “array_size” is the size of the array that we want to assign to the array.

• The array size is always mentioned inside the “[]”.

For example: Int age[5];

Initializing Arrays:

Initializing of array is very simple in c programming. The initializing values are enclosed within the curly braces in the declaration and placed following an equal sign after the array name. Here is an example which declares and initializes an array of five elements of type int.

Array can also be initialized after declaration.

int mark[5]={19,10,8,17,9}; It is not necessary to define the size of arrays during initialization

e.g. You can also initialize an array like this. int mark[]={19,10,8,17,9};

Accessing array elements

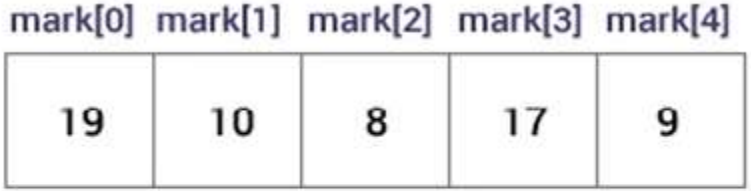
In C programming, arrays can be accessed and treated like variables in C.

For example:

```
scanf("%d",&mark[2]);
```

//statement to insert value in the third element of array mark[]

```
printf("%d",mark[2]);
```



Initialize an Array

Functions:

• A function is a block of code that performs a specific task. It has a name and it is reusable.It can be executed from as many different parts in a program as required, it can also return a value to calling program.

• All executable code resides within a function. It takes input, does something with it, then give the answer. A C program consists of one or more functions.

• A computer program cannot handle all the tasks by itself. It requests other program like entities called functions in C. We pass information to the function called arguments which specified when the function is called.

• A function either can return a value or returns nothing. Function is a subprogram that helps reduce coding.

Types of Function in C:

Library Functions in C

These functions are present in the c library and they are predefined.

- Example: sqrt() is a mathematical library function which is used for finding the square root of any number .The function scanf and printf() are input and output library function similarly we have strcmp() and strlen() for string manipulations.
- To use a library function we have to include some header file using the preprocessor directive #include.
- For example to use input and output function like printf() and scanf() we have to include stdio.h, for math library function we have to include math.h for string library string.h should be included.

User Defined Functions in C

A user can create their own functions for performing any specific task of program are called user defined functions. To create and use these function we have to know these 3 elements.

- Function Declaration
- Function Definition
- Function Call

Defining a Function:

The general form of a function definition in C programming language is as follows:

return_type function_nam e(parameter list)

{

body of the function

}

Category of Function based on argument and return type:

A function depending on whether arguments are present or not and whether a value is returned

or not may belong to any one of the following categories:

- Functions with no arguments and no return values.
- Functions with arguments and no return values.
- Functions with arguments and return values.
- Functions with no arguments and return values.

String:

String in C programming is a sequence of characters terminated with a null character ‘\0’. Strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a unique character ‘\0’.

Declaration of Strings:

Declaring a string is as simple as declaring a one-dimensional array. Below is the basic syntax for declaring a string.

```
char str_name[size];
```

C String Functions:

1)	strlen(string_name)	returns the length of string name.
2)	strcpy(destination, source)	copies the contents of source string to destination string.
3)	strcat(first_string, second_string)	concats or joins first string with second string. The result of the string is stored in first string.
4)	strcmp(first_string, second_string)	compares the first string with second string. If both strings are same, it returns 0.
5)	strrev(string)	returns reverse string.
6)	strlwr(string)	returns string characters in lowercase.
7)	strupr(string)	returns string characters in uppercase.

***Structure*:**

- Structure is a user-defined datatype in C language which allows us to combine data of different types together.
- Structure helps to construct a complex data type which is more meaningful.
- It is somewhat similar to an Array, but an array holds data of similar type only. But structure on the other hand, can store data of any type, which is practical more useful.

Syntax to Define a Structure in C

```
struct structName

{    // structure definition

 Data_type1 member_name1;

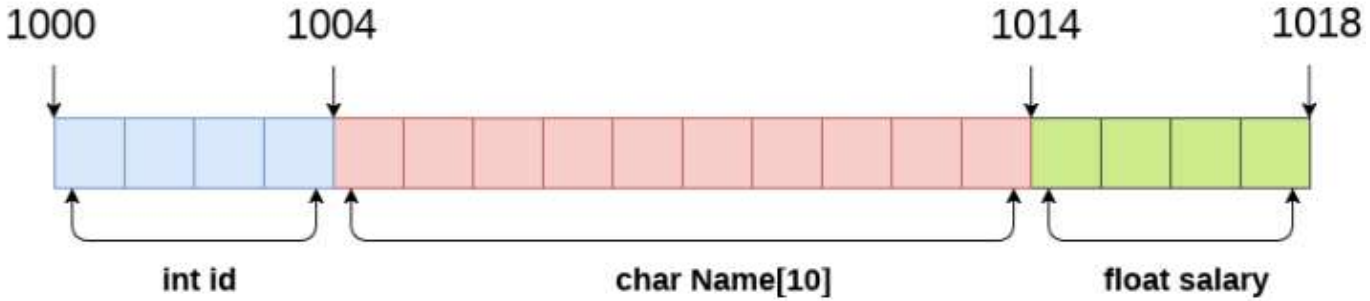
   Data_type2 member_name2;

   Data_type3 member_name3;

};
```

Example of Structure:

```
struct employee
{   int id;
    char name[10];
    float salary;
};
```



Declaring Structure Variables:

It is possible to declare variables of a structure, either along with structure definition or after the structure is defined. Structure variable declaration is similar to the declaration of any normal variable of any other datatype. Structure variables can be declared in following two ways:

1) Declaring Structure variables separately
2) Declaring Structure variables with structure definition

1) Declaring Structure variables separately:

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
};
struct Student S1, S2;      //declaring variables of struct Student
```

2) Declaring Structure variables with structure definition:

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
}S1, S2;
```

Accessing members of the structure:

There are two ways to access structure members:

1. By . (member or dot operator)
2. By -> (structure pointer operator)

Structure of Initialization:

```
struct Patient
{
    float height;
    int weight;
    int age;
};
struct Patient p1 = { 180.75 , 73, 23 };    //initialization
```

```
struct Patient p1;

p1.height = 180.75;      //initialization of each member separately

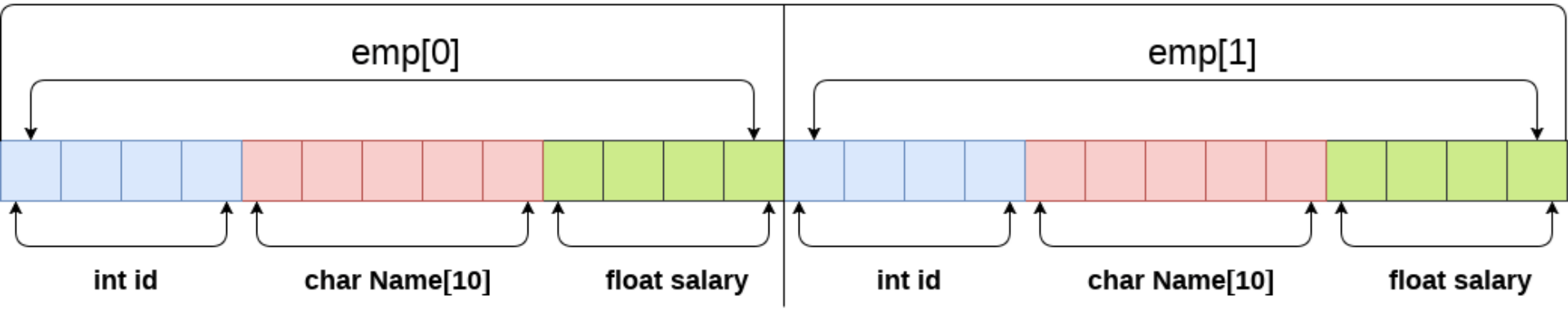
p1.weight = 73;

p1.age = 23;
```

Array of Structures in C:

- An array of structures in C can be defined as the collection of multiple structures variables where each variable contains information about different entities.
- The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.

Array of structures




```
struct employee
{
    int id;
    char name[5];
    float salary;
};
struct employee emp[2];
```

sizeof (emp) = 4 + 5 + 4 = 13 bytes

sizeof (emp[2]) = 26 bytes

C typedef:

- The **typedef** is a keyword that is used to provide existing data types with a new name. The C typedef keyword is used to redefine the name of already existing data types.
- When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes, which behave similarly to defining an alias for commands.

C typedef Syntax:

```
typedef existing_name alias_name;
```

After this declaration, we can use the *alias_name* as if it were the real *existing_name* in out C program.

Example of typedef in C:

#inlucde<stdio.h>

```
typedef int temprature;
Void main(){
        temperature var=20;
        printf(“%d”,var);
}
```

Use of typedef in C:

Following are some common uses of the typedef in C programming:

- The typedef keyword gives a meaningful name to the existing data type which helps other users to understand the program more easily.
- It can be used with structures to increase code readability and we don’t have to type struct repeatedly.
- The typedef keyword can also be used with pointers to declare multiple pointers in a single statement.
- It can be used with arrays to declare any number of variables.

typedef struct:

typedef can also be used with structures in the C programming language. A new data type can be created and used to define the structure variable.

Ex:

```
        Typedef struct student{
        Char name[20];
        Int roll no.
}Stud;
main(){
        Stud1 st;
}
```

```
struct student
{
char name[20];
int age;
};
typedef struct student stud;
stud s1, s2;
```

Union:

Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location. The union can also be defined as many members, but only one member can contain a value at a particular point in time.

Pointer:

A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address.

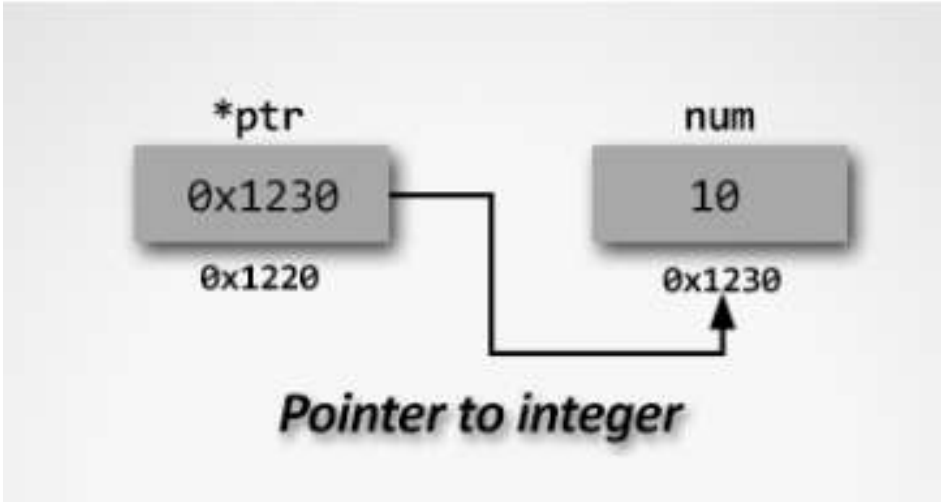
The general form of a pointer variable declaration is –

data_type *var_name;

Pointer allows various magical things to be performed in C:

- Pointers are more efficient in handling arrays and structures.
- Pointers are used to return multiple values from a function.
- Pointer allows dynamic memory allocation and deallocation (creation and deletion of variables at runtime) in C. Which undoubtedly is the biggest advantage of pointers.
- Pointer allows to refer and pass a function as a parameter to functions.

and many more....


*ptr
num
0x1230
10
0x1220
0x1230
Pointer to integer

Types of Pointers:

There are different types of pointers which are as follows –

- Null pointer.
- Void pointer
- Wild pointer.
- Dangling pointer

Null Pointer:

You create a null pointer by assigning the null value at the time of pointer declaration.

This method is useful when you do not assign any address to the pointer. A null pointer always contains value 0.

Void Pointer:

It is a pointer that has no associated data type with it. A void pointer can hold addresses of any type and can be typecast to any type.

It is also called a generic pointer and does not have any standard data type.

It is created by using the keyword void.

Wild Pointer:

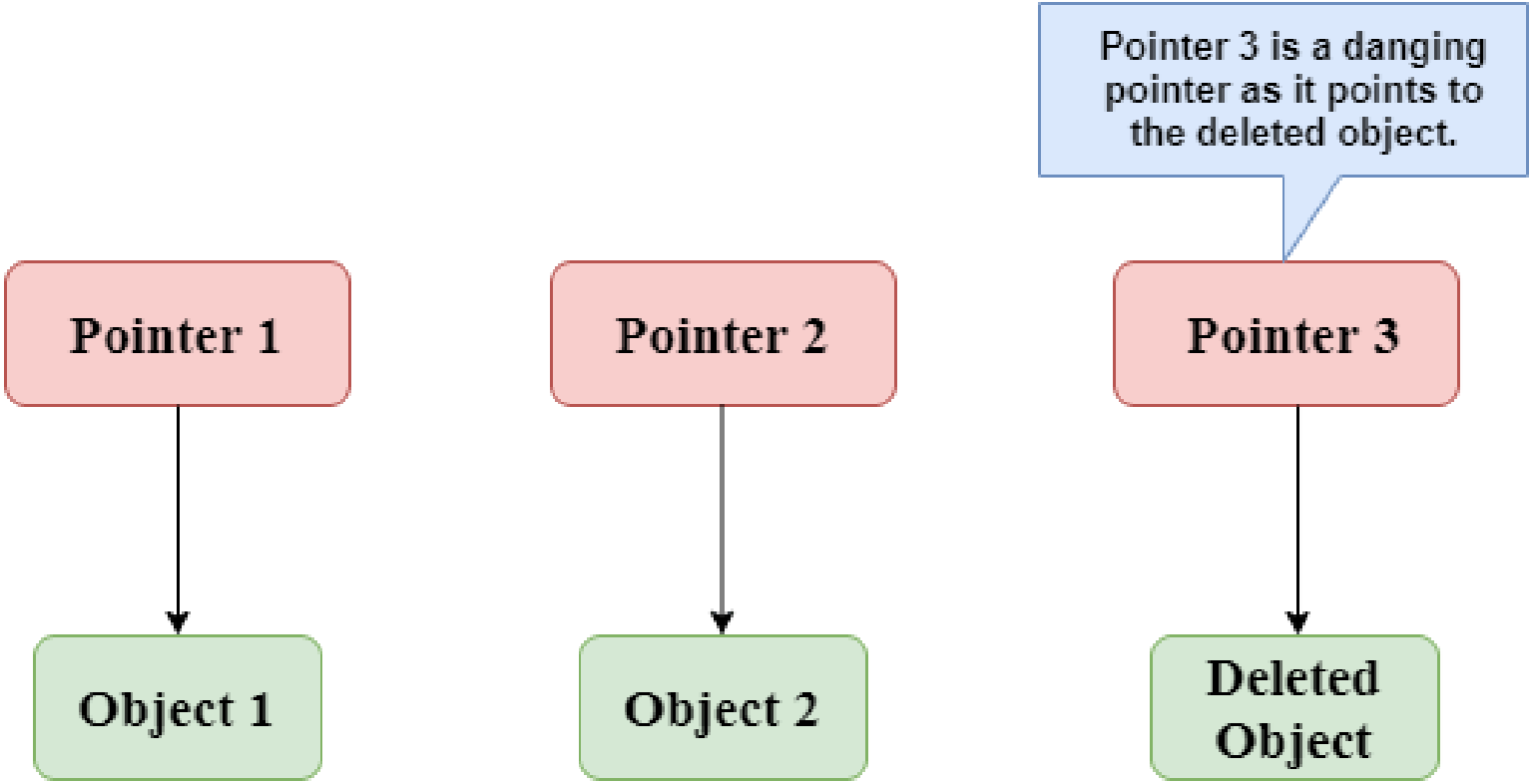
Wild pointers are also called uninitialized pointers. Because they point to some arbitrary memory location and may cause a program to crash or behave badly.

This type of C pointer is not efficient. Because they may point to some unknown memory location which may cause problems in our program. This may lead to the crashing of the program.

It is advised to be cautious while working with wild pointers.

Dangling Pointers in C:

- The most common bugs related to pointers and memory management is dangling/wild pointers. Sometimes the programmer fails to initialize the pointer with a valid address, then this type of initialized pointer is known as a dangling pointer in C.
- When we talk about allocation and deallocation of memory blocks, we see Dynamic Memory Allocation concepts. In Dynamic Memory Allocation, generally, we use malloc(), calloc() functions to allocate a memory block and free() function to deallocate a memory block in C Language. So, once we deallocate a memory block using the free() function, a Dangling Pointer is generated.


Pointer 3 is a danging pointer as it points to the deleted object.
Pointer 1
Pointer 2
Pointer 3
Object 1
Object 2
Deleted Object

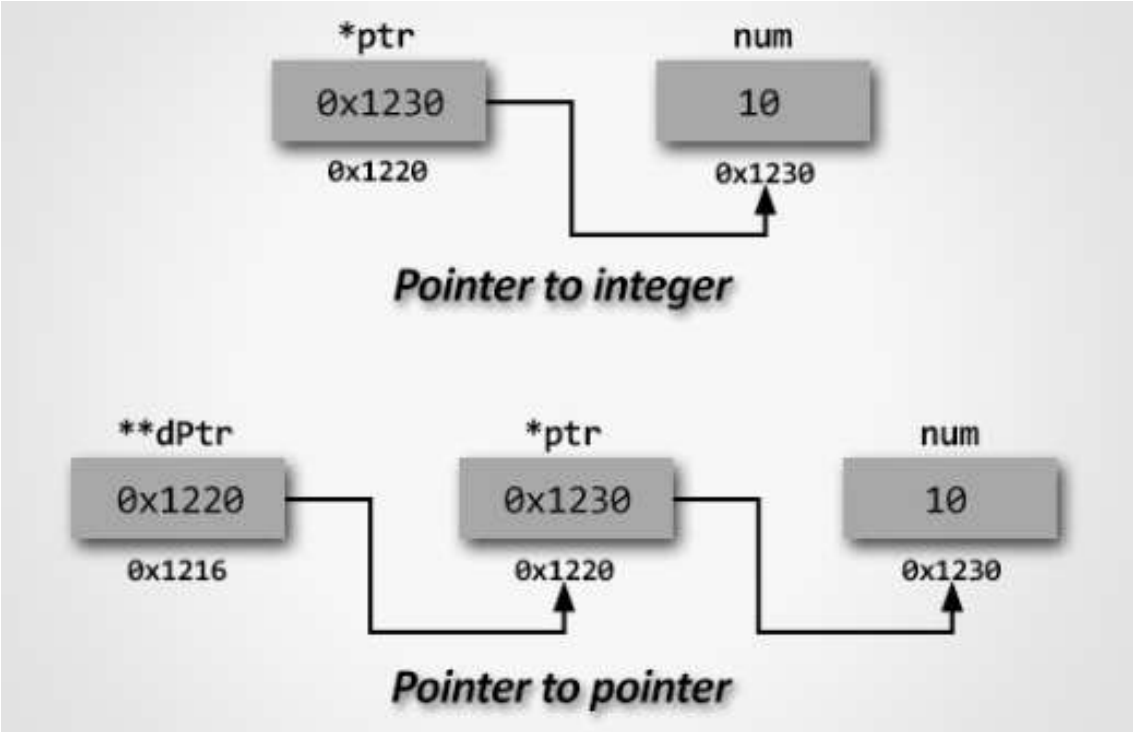
Double Pointer:

Pointers are used to store the address of other variables of similar datatype. But if you want to store the address of a pointer variable, then you again need a pointer to store it. Thus, when one pointer variable stores the address of another pointer variable, it is known as Pointer to Pointer variable or Double Pointer.

Syntax:

```
int **var_name;
```

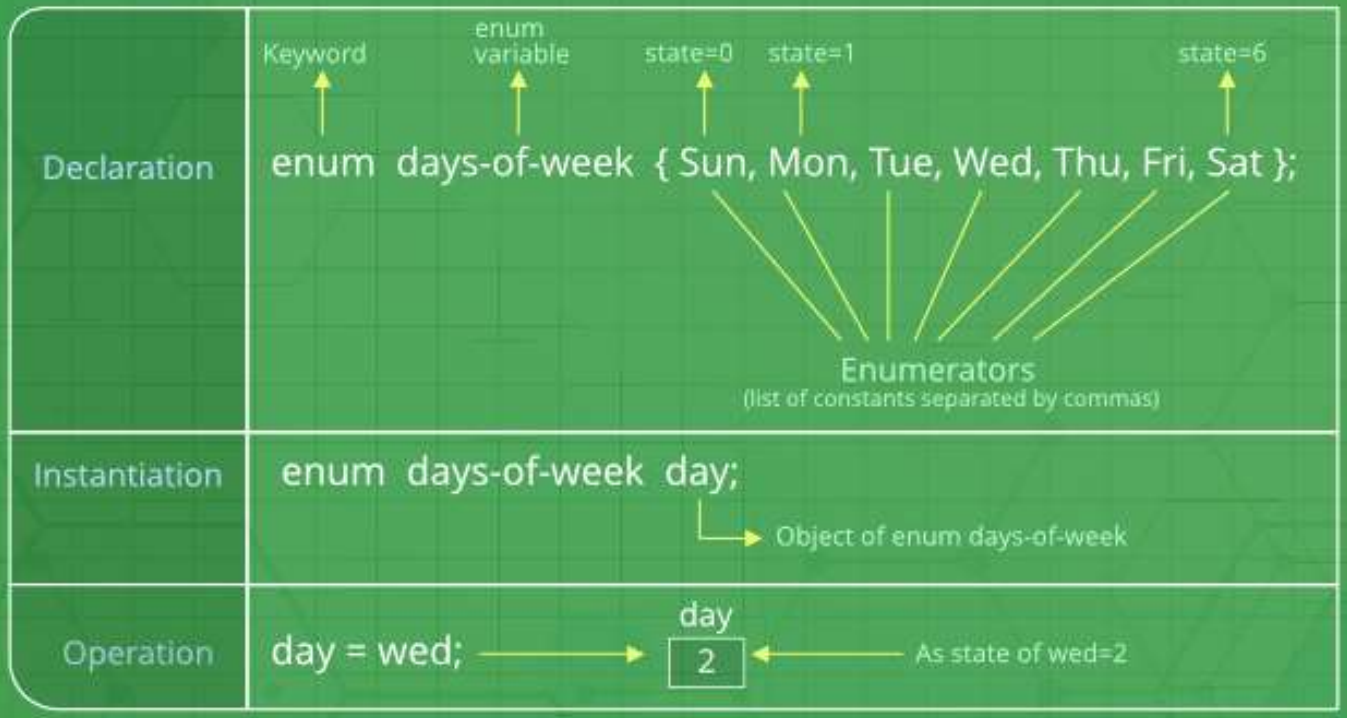
DOUBLE POINTER:



ENUMERATIONS:

- Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.
- An enum is a special type that represents a group of constants (unchangeable values).
- To create an enum, use the `enum` keyword, followed by the name of the enum, and separate the enum items with a comma:

Enum in C


Declaration
Keyword
enum variable
state=0
state=1
state=6
enum days-of-week { Sun, Mon, Tue, Wed, Thu, Fri, Sat };
Enumerators
(list of constants separated by commas)
Instantiation
enum days-of-week day;
Object of enum days-of-week
Operation
day = wed;
day
2
As state of wed=2

Enumerated Type Declaration:

```
// In both of the below cases, "day" is
// defined as the variable of type week.
enum week{Mon, Tue, Wed};
enum week day;
// Or
  enum week{Mon, Tue, Wed}day;
```

Why And When To Use Enums?

Enums are used to give names to constants, which makes the code easier to read and maintain.

Use enums when you have values that you know aren't going to change, like month days, days, colors, deck of cards, etc.

Dynamic memory allocation in C:

The concept of **dynamic memory allocation in c language** *enables the C programmer to allocate memory at runtime*. Dynamic memory allocation in c language is possible by 4 functions of stdlib.h header file.

1. malloc()
2. calloc()
3. realloc()
4. free()

Methods used for dynamic memory allocation:

malloc()	allocates single block of requested memory.
calloc()	allocates multiple block of requested memory.
realloc()	reallocates the memory occupied by malloc() or calloc() functions.
free()	frees the dynamically allocated memory.

malloc() function in C:

The malloc() function allocates single block of requested memory.

It doesn't initialize memory at execution time, so it has garbage value initially.

It returns NULL if memory is not sufficient.

The syntax of malloc() function is given below:

```
ptr=(cast-type*)malloc(n*byte-sizeof(cast_type))
```

calloc() function in C:

The calloc() function allocates multiple block of requested memory.

It initially initialize all bytes to zero.

It returns NULL if memory is not sufficient.

The syntax of calloc() function is given below:

```
ptr=(cast-type*)calloc(number, byte-sizeof(cast_type))
```

realloc() function in C:

If memory is not sufficient for malloc() or calloc(), you can reallocate the memory by realloc() function. In short, it changes the memory size.

Let's see the syntax of realloc() function.

```
ptr=(cast_type*)realloc(ptr, new-size*sizeof(cast_type))
```

free() function in C:

The memory occupied by malloc() or calloc() functions must be released by calling free() function. Otherwise, it will consume memory until program exit.

Let's see the syntax of free() function.

1. free(ptr)

Storage classes:

Variables in C differ in behavior. The behavior depends on the storage class a variable may assume. From C compiler‟s point of view, a variable name identifies some physical location within the computer where the string of bits representing the variable‟s value is stored.

There are four storage classes in C:

- Automatic storage class
- Extern storage class
- Static storage class
- Register storage class

Automatic Storage Class:

The features of a variable defined to have an automatic storage class are as under:

Keyword: Auto

Storage: Memory.

Default initial value: An unpredictable value, which is often called a garbage value.

Scope: Local to the block in which the variable is defined.

Life: Till the control remains within the block in which the variable is defined.

Extern Storage Class:

The features of a variable whose storage class has been defined as external are as follows:

Keyword	Extern
Storage	Memory
default initial value	Zero
Scope	Global
Life	As long as the program execution does not come to end

External variables differ from those we have already discussed in that their scope is global, not local. External variables are declared outside all functions, yet are available to all functions that care to use them. Here is an example to illustrate this fact.

Ex:

```
#include<stdio.h>

extern int i;

void main() {

printf(“i=%d”,i);

}
```

Static Storage Class:

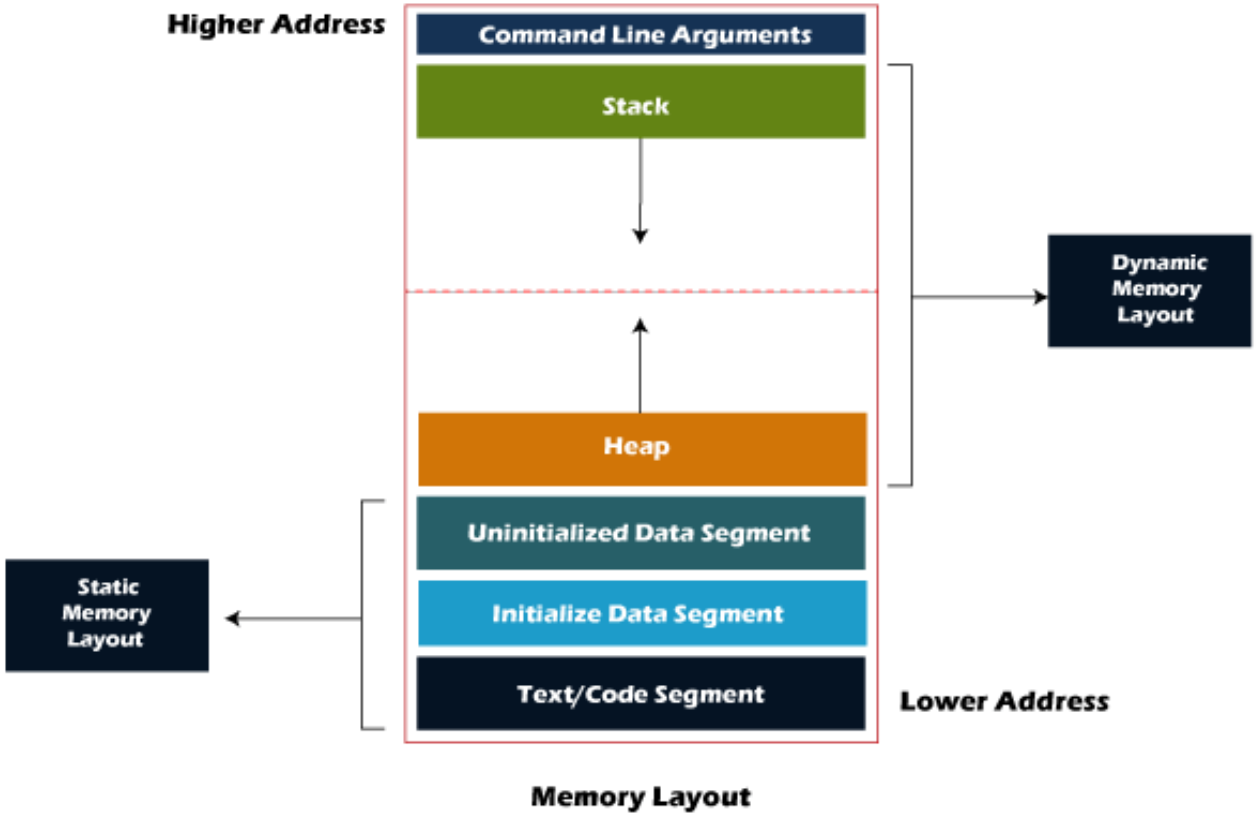
Keyword	Static
Storage	Memory.
Default initial value	Zero.
Scope	Local to the block in which the variable is defined.
Life	Value of the variable persists between different function calls

Register Storage Class:

A value stored in a CPU register can always be accessed faster than the one that is stored in memory. Therefore, if a variable is used at many places in a program it is better to declare its storage class as register.

Keyword	Register
Storage	CPU Registers
default initial value	An unpredictable value, which is often called a garbage value.
Scope	Local to the block in which the variable is defined.

Memory Management:



File Handling in C:

- Files are used to store data permanently on hard disk. C programming supports built in library function to interact with files and directories.
- In programming, we may require some specific input data to be generated several numbers of times. Sometimes, it is not enough to only display the data on the console. The data to be displayed may be very large, and only a limited amount of data can be displayed on the console, and since the memory is volatile, it is impossible to recover the programmatically generated data again and again.
- However, if we need to do so, we may store it onto the local file system which is volatile and can be accessed every time. Here, comes the need of file handling in C.

Why do we need File Handling in C?

Operations using the C program are done on a prompt/terminal which is not stored anywhere. The output is deleted when the program is closed. But in the software industry, most programs are written to store the information fetched from the program. The use of file handling is exactly what the situation calls for.

- **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

File Pointer in C:

A file pointer is a reference to a particular position in the opened file. It is used in file handling to perform all file operations such as read, write, close, etc. We use the **FILE** macro to declare the file pointer variable. The FILE macro is defined inside **<stdio.h>** header file.

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file

Opening File: fopen()

We must open a file before it can be read, write, or update. The fopen() function is used to open a file. The syntax of the fopen() is given below.

1. **FILE** *fopen(**const char** * filename, **const char** * mode);

The fopen() function accepts two parameters:

- The file name (string). If the file is stored at some specific location, then we must mention the path at which the file is stored.
- The mode in which the file is to be opened. It is a string.

Mode	Description
r	opens a text file in read mode
w	opens a text file in write mode
a	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode
a+	opens a text file in read and write mode

Closing File: fclose():

The fclose() function is used to close a file. The file must be closed after performing all the operations on it. The syntax of fclose() function is given below:

Syntax:

```
fclose( FILE_POINTER );
```

Write To a File:

The `w` mode means that the file is opened for writing. To insert content to it, you can use the `fprint()` function and add the pointer variable (`fptr` in our example) and some text:

Syntax:

fprintf(FILE *stream, const char *format [, argument, ...]);

Reading File : fscanf() function

The fscanf() function is used to read set of characters from file. It reads a word from the file and returns EOF at the end of file.

Syntax:

```
fscanf(FILE *stream, const char *format [, argument, ...])
```

Append Content To a File:

If you want to add content to a file without deleting the old content, you can use the `a` mode.

The `a` mode appends content at the end of the file:

Example:

```
fptr = fopen("demo.txt", "a");//Open file in append mode
```

C fputc() and fgetc()

Writing File : fputc() function:

The fputc() function is used to write a single character into file. It outputs a character to a stream.

Syntax:

```
fputc(int c, FILE *stream)
```

Reading File : fgetc() function

The fgetc() function returns a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

Syntax:

```
int fgetc(FILE *stream)
```

C fputs() and fgets()

Writing File : fputs() function

The fputs() function writes a line of characters into file. It outputs string to a stream.

Syntax:

```
int fputs(const char *s, FILE *stream)
```

Reading File : fgets() function

The fgets() function reads a line of characters from file. It gets string from a stream.

Syntax:

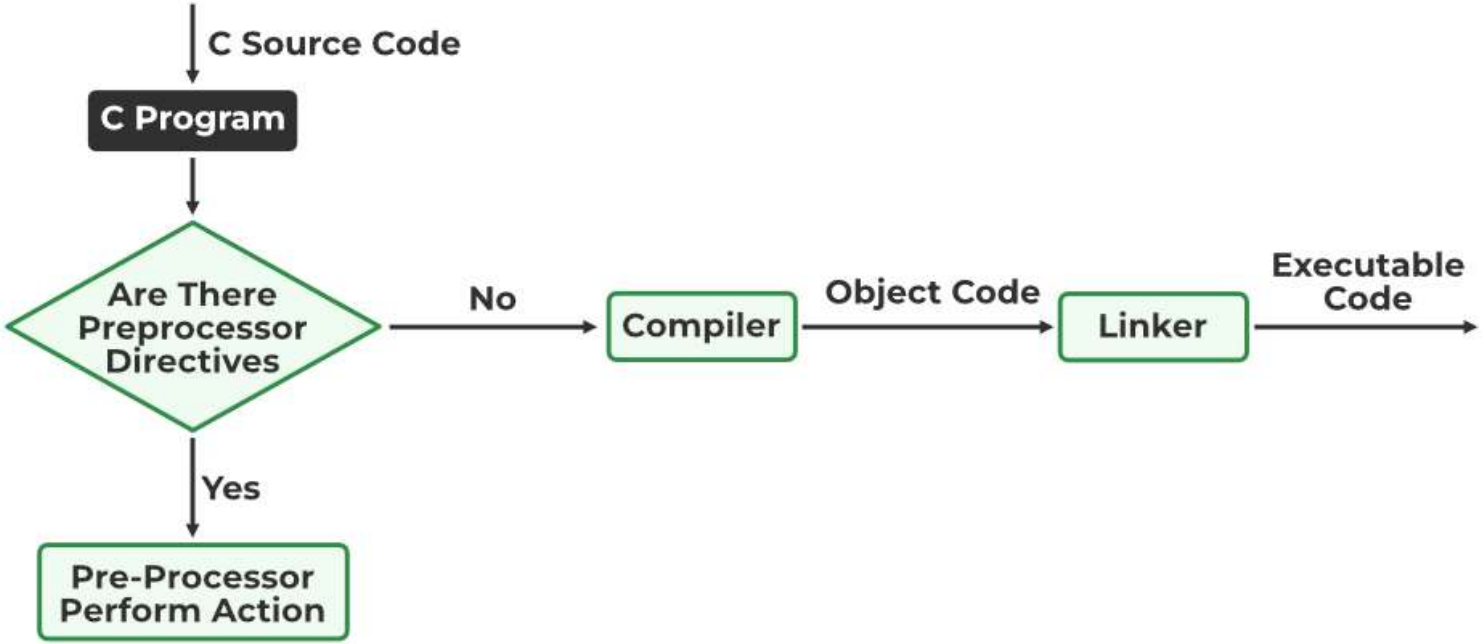
```
char* fgets(char *s, int n, FILE *stream)
```

What are Preprocessor Directives:

Whenever we write a certain code in C language, it goes through the process of compilation, where it gets converted from source code to machine-understandable code.

But before the compilation process, the source code goes through preprocessing, which is done by the preprocessor.

Processor in C



Macros in C language?

The macro in C language is known as the piece of code which can be replaced by the macro value. The macro is defined with the help of #define preprocessor directive and the macro doesn’t end with a semicolon(;). Macro is just a name given to certain values or expressions it doesn't point to any memory location.

Whenever the compiler encounters the macro, it replaces the macro name with the macro value. Two macros could not have the same name.

The syntax of the macro is as shown in the following figure. Here, we will have the three components:

1) #define
2) macro_name
3) Macro_Value

Ex: #define Macro_name Macro_value

C #ifdef:

The #ifdef preprocessor directive checks if macro is defined by #define. If yes, it executes the code otherwise #else code is executed, if present.

Syntax:

```
#ifdef MACRO

//code

#endif
```

Syntax with #else:

```
#ifdef MACRO

//successful code

#else

//else code

#endif
```

C #if and #else:

The #if preprocessor directive evaluates the expression or condition. If condition is true, it executes the code otherwise #elseif or #else or #endif code is executed.

Syntax:

```
#if expression

//Statement

#endif
```

```
#if expression

//Statement

#else

//Statement

#endif
```

What is a Volatile Keyword in C?

- In C, the volatile keyword is a qualifier that can be applied to a variable declaration. When a variable is declared as volatile, it indicates to the compiler that the variable’s value may change at any time without any action being taken by the code the compiler finds nearby.
- This is particularly useful in scenarios where the variable can be modified externally, such as by hardware, an interrupt service routine, or concurrently running threads.

Syntax of Volatile Keyword in C:

```
volatile data_type variable_name;
```

For example, consider a program that reads data from a hardware device and stores it in a variable. Without the volatile keyword, the compiler may assume that the value of the variable will not change during the program's execution, and it may optimize the code by reading the value of the variable only once.

However, the value of the variable may change unexpectedly due to external factors, such as a change in the hardware device's state. In this case, using the volatile keyword ensures that the program reads the current value of the variable every time it is accessed, rather than using any cached or optimized value